

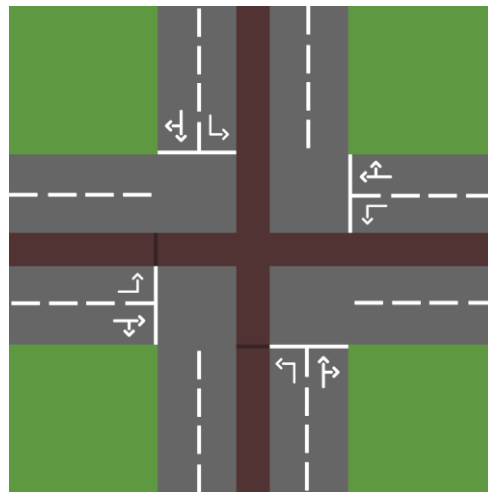
Primjena DQN i PPO algoritama za upravljanje saobraćajem na raskrsnici sa željezničkim prugom

Andrej Vujić, RA130/2023

3. jun 2026.

1. Definicija problema

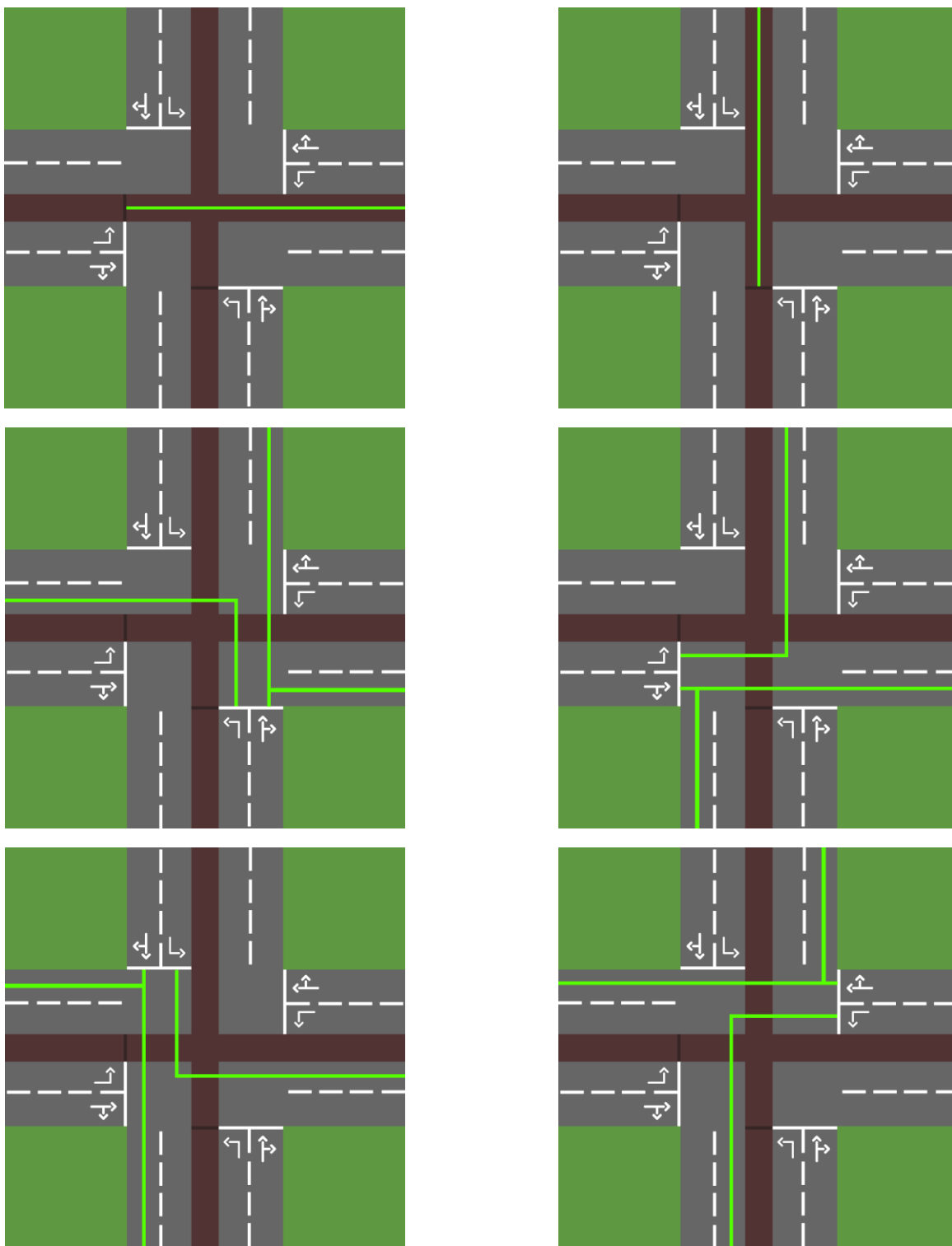
Cilj projekta je razvoj pojednostavljenog 2D simulacionog okruženja u kojem agent pomoću **učenja potkrepljivanjem** uči da kontroliše saobraćaj na raskrsnici. Agent treba da kontroliše semafore za automobile i signale dozvole za prolaz vozova, tako da se izbjegnu sudari i da se minimizuje vrijeme čekanja.



Izgled okruženja

Problem upravljanja saobraćajem je još uvijek otvoreno pitanje i aktivno se radi na povećavanju efikasnosti i bezbjednosti saobraćaja. Primjena sistema bez inteligencije koji upravljaju semaforima mijenjajući dozvole kretanja periodično, u fiksnim intervalima, može biti prilično neefikasno i dovesti do stvaranja kolona, povećanja potrošnje goriva, a samim tim i većeg zagađenja okoline. S druge strane je faktor bezbjednosti, ukoliko automobili moraju preći preko željezničke pruge neophodno je dozvoliti prolaz samo u slučajevima kada je to bezbjedno. Vozovi su znatno teži od automobila i mnogo više vremena im je potrebno da se zaustave, te ponovo dostignu maksimalnu brzinu nakon zaustavljanja. Zbog toga im je neophodno obezbijediti prioritet u raskrsnicama. Očekivani

rezultat projekta je agent koji razumije način funkcionisanja raskrsnice i može da obezbjedi bezbjedan, visok protok automobila uz maksimalnu efikasnost za vozove.



Putanje automobila i vozova

2. Skup podataka

Za ovaj projekat nije potreban unaprijed pripremljen skup podataka, jer se koristi reinforcement learning. Agent uči iz iskustva kojeg dobija kroz interakciju sa okruženjem. Svaki korak simulacije proizvodi iskustvo koje sadrži sledeće informacije: [stanje, akcija, nagrada, novo stanje, flag za kraj epizode]. Agent na osnovu stanja okruženja donosi odluku o tome šta treba da bude njegov sledeći potez.

Stanje okruženja može sadržati:

- broj vozila koja čekaju iz svakog pravca,
- ukupno vrijeme čekanja vozila,
- informaciju da li postoji voz koji treba da prođe kroz raskrsnicu i koja je njegova udaljenost od raskrsnice

3. Metodologija

Prvi dio projekta obuhvata razvoj 2D simulacionog okruženja. Okruženje će biti implementirano u programskom jeziku Python, dok će se za vizualizaciju koristiti pygame. Vozila će se kretati po unaprijed definisanim putanjama, zaustavljati se ispred crvenog svetla ili zatvorenog prelaza i nastavljati kretanje kada dobiju dozvolu za prolaz.

U projektu će ručno biti implementirana dva algoritma, **Deep Q Network** (DQN) i **Proximal Policy Optimization** (PPO).

DQN je off-policy algoritam koji koristi neuronsku mrežu za aproksimaciju Q-vrijednosti (vrijednost koja aproksimira očekivanu ukupnu nagradu za neko stanje). Agent bira akciju pomoću epsilon-greedy strategije, čime se postiže balans između istraživanja i korišćenja naučene strategije s ciljem postizanja boljeg rezultata.

PPO je on-policy algoritam koji koristi dvije mreže. Actor mreža definiše politiku (način na koji se bira akcija koja se primjenjuje u određenom stanju okruženja), dok critic mreža procjenjuje vrijednost stanja, odnosno vrši evaluaciju akcija. PPO ograničava prevelike promjene politike, što doprinosi stabilnosti treninga.

Za oba algoritma biće korišten diskretan skup akcija.

Za DQN skup akcija je diskretan skup od 11 akcija: [allow_north_south, allow_south_north, allow_west_east, allow_east_west, allow_north_south_turn, allow_south_north_turn, allow_west_east_turn, allow_east_west_turn, allow_train_south_north, allow_train_east_west, stop_all].

Za PPO skup akcija predstavlja niz od 10 binarnih vrijednosti (akcije su iste kao DQN, bez stop_all). Izlaz iz mreže je 10 brojeva iz intervala [0, 1] koji predstavljaju vjerovatnoću za omogućavanje kretanja. Akcija se formira tako što se formira 10 Bernulijevih raspodjela gdje je vjerovatnoća za omogućavanje kretanja p (izlaz iz mreže),

a vjerovatnoća zabrane kretanja $1 - p$, i iz svake izabere po jedan broj koji predstavlja odluku agenta za taj dio raskrsnice.

Akcije za dozvolu kretanja na pravcu i skretanje na tom pravcu su namjerno razdvojene s ciljem da agent sam nauči na kojim dijelovima raskrsnice može dozvoliti kretanje u paraleli.

PPO u ovakvoj postavci ima značajno veću slobodu od DQN algoritma jer u jednom koraku može promijeniti stanje više dijelova raskrsnice. DQN je ograničen na jednu akciju po koraku zbog toga što bi u slučaju omogućavanja kombinacija imao skup od 2^{11} mogućih akcija. Ostaje da vidimo kako će to uticati na rezultate.

Funkcija za dodjeljivanje nagrada će uzimati u obzir broj vozila koja su prošla kroz raskrsnicu, broj vozila koja čekaju, vrijeme čekanja vozila, da li je voz morao da stane i da li je došlo do sudara. Epizoda se završava ako dođe do sudara ili ako istekne predefinisani broj koraka.

4. Način evaluacije

Evaluacija agenata će se vršiti po više kriterijuma:

1. Bezbjednost – prosječan broj sudara
2. Efikasnost saobraćaja – prosječno i maksimalno vrijeme čekanja, prosječna i maksimalna dužina kolone, protok vozila kroz raskrsnicu
3. Poštovanje prioriteta vozova – prosječno i maksimalno čekanje vozova
4. Stabilnost ponašanja – vrijeme treniranja, promjena nagrade kroz epizode, broj promjena stanja semafora u epizodi

Očekuje se da će oba algoritma imati bolje mjere evaluacije od random agenta čiji su potezi u potpunosti nasumični i agenta koji predstavlja klasičnu raskrsnicu sa semaforima koji periodično, unaprijed definisanim redoslijedom, dozvoljavaju kretanje iz svakog pravca.

5. Tehnologije

Za realizaciju projekta koristiće se programski jezik Python i biblioteke pygame, PyTorch, numpy, matplotlib.

6. Primjeri gotovih i srodnih rješenja

Srodna rješenja uključuju reinforcement learning okruženja za upravljanje saobraćajem koja se mogu pronaći unutar biblioteke [Gymnasium](#). Navedena rješenja koriste biblioteke

i alate za simulaciju saobraćaja kao što je [SUMO](#). U ovom projektu neće se koristiti gotovo okruženje, već će biti implementirana vlastita pojednostavljena 2D simulacija.

Postojeće implementacije DQN i PPO algoritama u bibliotekama poput [StableBaselines3](#) i [RLlib](#) neće biti direktno kopirane, već će služiti kao teorijska i tehnička referenca.

7. Relevantna literatura

1. GeeksForGeeks - [Deep Q Learning](#) i [Proximal Policy Optimization](#)
2. YouTube - [Reinforcement Learning Introduction](#), [DQN Explanation](#) i [PPO Explanation](#)
3. [Playing Atari with Deep Reinforcement Learning](#)
4. [Proximal Policy Optimization Algorithms](#)
5. Python, PyTorch, Matplotlib i numpy dokumentacija